

LAB GUIDELINES

Lab 2 — Storage Deployment: RAID 0/1/5/6/10 with mdadm

Maps to CompTIA Server+ objective 1.2 — **Given a scenario, deploy and manage storage** (RAID levels 0, 1, 5, 6, 10; JBOD; hardware vs. software; capacity planning).

You will build every RAID level the exam tests, on **loopback files** that behave like real disks inside the KillerCoda VM.

Run all commands on <https://killercoda.com/playgrounds/scenario/ubuntu>

Required software (free, apt): mdadm

```
apt update && apt install -y mdadm
```

Step 1 — Create six "disks" with loopback files

```
mkdir -p /srv/disks && cd /srv/disks
for i in 1 2 3 4 5 6; do truncate -s 200M disk$i.img; losetup /dev/loop$i disk$i.img; done
losetup -a
```

Each /dev/loopN now behaves exactly like a /dev/sdX drive for mdadm.

Step 2 — RAID 0 (stripe, no redundancy — performance only)

```
mdadm --create /dev/md0 --level=0 --raid-devices=2 /dev/loop1 /dev/loop2
mkfs.ext4 /dev/md0 && mkdir -p /mnt/r0 && mount /dev/md0 /mnt/r0
df -h /mnt/r0 # ~400 MB usable (2 × 200 MB, 0 % overhead)
```

Capacity rule: **N × disk**. Fault tolerance: **0**.

Step 3 — RAID 1 (mirror — survives 1 disk failure)

```
mdadm --stop /dev/md0
mdadm --create /dev/md1 --level=1 --raid-devices=2 /dev/loop1 /dev/loop2
cat /proc/mdstat # watch resync
```

Capacity rule: **1 × disk** (50 % overhead). Tolerates **1 drive** failure.

Step 4 — RAID 5 (single parity)

```
mdadm --stop /dev/md1
mdadm --create /dev/md5 --level=5 --raid-devices=3 /dev/loop1 /dev/loop2 /dev/loop3
mdadm --detail /dev/md5 | grep -E "Array Size|Used Dev Size"
```

Capacity rule: **(N – 1) × disk**. Tolerates **1 drive** failure. Write-intensive workloads suffer due to parity recalculation.

Step 5 — RAID 6 (double parity)

```
mdadm --stop /dev/md5
mdadm --create /dev/md6 --level=6 --raid-devices=4 /dev/loop1 /dev/loop2 /dev/loop3 /dev/loop4
```

Capacity rule: **(N – 2) × disk**. Tolerates **2 drives** failing simultaneously — recommended for large SATA arrays where rebuild times are long.

Step 6 — RAID 10 (stripe of mirrors)

```
mdadm --stop /dev/md6
mdadm --create /dev/md10 --level=10 --raid-devices=4 /dev/loop1 /dev/loop2 /dev/loop3 /dev/loop4
```

Capacity rule: **N / 2**. Tolerates up to **1 drive per mirror** failing. Best blend of performance + redundancy — preferred for databases.

Step 7 — JBOD (just a bunch of disks)

```
mdadm --stop /dev/md10
mdadm --create /dev/md0 --level=linear --raid-devices=2 /dev/loop5 /dev/loop6
```

No redundancy, no striping — disks are concatenated. Used when you want one large logical volume without spending overhead on parity.

Step 8 — Simulate a failure and rebuild

```
mdadm --create /dev/md1 --level=1 --raid-devices=2 /dev/loop1 /dev/loop2
mdadm /dev/md1 --fail /dev/loop2 --remove /dev/loop2
mdadm /dev/md1 --add /dev/loop3
watch -n1 cat /proc/mdstat # observe resync %
```

Note the rebuild time — this is why RAID 6 wins for very large disks (longer single-disk rebuilds mean a higher chance of a second failure during rebuild).

Step 9 — Hardware vs. software RAID

Factor	Hardware RAID	Software RAID (`mdadm`)
CPU cost	None (on controller)	Host CPU
Battery-backed cache	Yes (BBWC / FBWC)	No (rely on UPS + journaling)
Portability	Tied to controller firmware	Any Linux host
Cost	Hundreds of \$	Free

Free online tools

- **RAID calculator (free)** — <https://www.raid-calculator.com/>
- **mdadm cheat sheet** — https://raid.wiki.kernel.org/index.php/Linux_Raid

What you learned

- Capacity and fault-tolerance formulas for RAID 0, 1, 5, 6, 10 and JBOD.
- Building, failing, and rebuilding arrays with `mdadm`.
- Why RAID 6 beats RAID 5 for large drives, and when RAID 10 beats both.
- The trade-offs between hardware RAID and software RAID for a Server+ deployment decision.